Project Overview

Created	March 6, 2026 5:56 PM
Class	CISC 844
Type	
Reviewed	☐

Table of Contents:

Proposal

Implementation Ideas

Final Report

Presentation

Model-Driven Project Concept: DSL for eCherry Reactor Modeling

Summary: This project introduces a domain-specific modeling language for electrochemical reactor design that automatically generates simulation models using the eCherry framework.

1. Overall Model-Driven Pipeline

Our project follows the standard Model-Driven Engineering Pipeline

```
Domain Problem
      ↓
  Metamodel
      ↓
```

```
    Models
      ↓
Model Transformation / Automation
      ↓
Generated Artifacts (eCherry simulation models)
```

This structure ensures that the project clearly demonstrates the core concepts of model-driven software development

2. Domain Problem

Electrochemical reactor simulations using **eCherry** require manually assembling and configuring multiple **Modelica components**.

This process:

- requires knowledge of **Modelica**
- requires understanding **eCherry component structure**
- can be **time-consuming and error-prone**

Problem Statement

> Creating electrochemical reactor simulations with eCherry requires manually assembling and configuring low-level Modelica components, which can be complex and error-prone for domain experts. This project introduces a higher-level modeling language that automatically generates eCherry simulation models.

Who This Helps

This tool would primarily help:

- **researchers working with electrochemical reactors**
- **engineers designing reactor configurations**
- domain experts who understand **chemistry or electrochemistry but not Modelica**

Instead of writing Modelica models directly, users will describe reactors using a **higher-level modeling language**.

3. Domain

The project domain is:

Electrochemical reactor modeling

Examples include:

- electrolyzers
- fuel cells
- electrochemical reactors

These systems typically consist of components such as:

- electrodes
- electrolytes
- separators
- flow channels

This aligns well with the **component-based architecture used in eCherry**.

4. Metamodel

We define a **metamodel** describing the structure of electrochemical reactor systems.

Example Metamodel Structure

```
Reactor
 ├ Electrode
 ├ Electrolyte
 ├ Separator
 ├ FlowChannel
```

```
OperatingConditions
 ├ Temperature
 ├ Pressure
 └ FlowRate
```

This metamodel defines the **core entities and relationships** in the domain.

Purpose of the Metamodel

The metamodel:

- defines the **structure of valid reactor models**
- provides the **foundation for the DSL**
- demonstrates **abstraction and domain modeling**

5. Domain-Specific Language (DSL)

The DSL serves as the **user-facing modeling language**.

Instead of writing Modelica code, users define reactors at a higher level of abstraction.

Example DSL

```
reactor Electrolyzer {
    electrode_area = 10 cm^2
    electrolyte = KOH
    temperature = 298K
}
```

This DSL represents a **model instance** of the metamodel.

6. Model Instances

We will create several **example models** to demonstrate the system.

Examples may include:

- Small laboratory electrolyzer
- Industrial-scale reactor
- Multi-cell electrochemical reactor

These examples will show how the DSL can represent **different reactor configurations**.

7. Model Transformation / Automation

The key model-driven component of the project is the **model transformation pipeline**.

```
DSL Model
      ↓
Parser
      ↓
Internal Model Representation
      ↓
Transformation Rules
      ↓
Generated eCherry Modelica Model
```

This transformation demonstrates:

- **model transformation**
- **automation of system design**

Users will not need to manually write Modelica models.

8. Generated Artifacts

The output of the transformation process will be:

eCherry Modelica simulation models

These generated models can then be executed in a **Modelica simulation environment**.

This demonstrates the model-driven concept:

```
Model → Executable Artifact
```

9. Automation

Automation appears in two ways.

1. Automatic Model Generation

The system automatically generates **eCherry simulation models** from DSL descriptions.

2. Parameterized Reactor Design

Users can modify parameters and regenerate models.

Example:

```
change electrode_area
      ↓
regenerate simulation model
```

This enables **rapid exploration of reactor configurations**.

10. Abstraction

The core contribution of the project is **abstraction**.

Instead of requiring users to interact with low-level simulation components:

```
High-Level Reactor Model
          ↓
Transformation
          ↓
Low-Level eCherry Model
```

The DSL acts as an **abstraction layer** over the eCherry modeling framework.

11. Prototype Tool

We will implement a **prototype tool** that includes:

- DSL parser
- model generator
- transformation scripts

This can be implemented using **Python** or another lightweight toolchain.

The prototype will demonstrate the full modeling pipeline.

12. Evaluation

We will evaluate the system by comparing:

Manual Model Creation

- manual Modelica code
- higher complexity

DSL-Based Model Creation

- shorter model descriptions
- faster design iteration

Possible metrics:

- model correctness
- modeling effort
- model complexity
- lines of code
- design iteration speed

13. Demonstration

For the presentation we can demonstrate the workflow:

```
1. Write DSL reactor model
2. Run generator
3. Generate eCherry Modelica model
4. Run simulation
```